HPC-Driven Predictive Quality Platform

Technical Documentation & Architecture Guide

Proof of Concept — Battery Thermal Management

June 11, 2026

Version 1.0 | Confidential

Table of Contents

1. Executive Summary

This document describes the **HPC-Driven Predictive Quality Platform** — a functional prototype of a SaaS-style predictive failure-detection system designed for Original Equipment Manufacturers (OEMs). The platform leverages High-Performance Computing (HPC) digital twin simulations to detect early warning signs of vehicle subsystem failures before they result in costly warranty claims.

The proof-of-concept focuses on the **Battery Thermal Management** subsystem in electric vehicles, where a single pack replacement can exceed $10,000 per vehicle. By identifying degradation patterns 24–72 hours before failure, the platform enables proactive maintenance and delivers a projected **30–40% reduction in warranty validation cycles**.

The platform is service-oriented from day one, allowing additional subsystems (HVAC, infotainment, powertrain) to be plugged in incrementally without modifying the dashboard, API layer, or predictive models.

2. Platform Overview

2.1 Core Purpose & Business Value

The platform serves as a **predictive maintenance intelligence layer** that sits between vehicle telemetry streams and OEM service operations. Its core capabilities include:

- **Real-time drift detection** — Identifies gradual degradation patterns using EWMA statistical methods
- **Failure probability estimation** — Transparent logistic models predict failure likelihood with 95% confidence intervals
- **Time-to-failure forecasting** — Heuristic-based horizon estimation to prioritize service actions
- **Automated alerting** — De-duplicated, severity-based notifications with acknowledgement workflow
- **A/B model testing** — Hash-deterministic routing enables honest comparison of model versions
- **ROI quantification** — Real-time warranty savings projections for executive stakeholders

2.2 Target Subsystem: Battery Thermal Management

Modern EV battery packs stream the richest telemetry available on the vehicle CAN bus: cell temperatures, coolant flow rates, pack current, voltage, and State of Charge (SoC). Thermal-related events drive the **largest single warranty cost line** for EV OEMs.

Key telemetry signals monitored:

Signal	Unit	Source	Purpose
Max Cell Temperature	°C	CAN Bus	Pack thermal state indicator
Coolant ΔT (out – in)	°C	Sensors	Cooling system effectiveness
Coolant Flow Rate	L/min	Flow sensor	Pump health proxy
Pack Current	A	BMS	Thermal load driver
State of Charge	%	BMS	Operating context

Table 1: Primary telemetry signals for Battery Thermal Management monitoring.

3. System Architecture

3.1 Service-Oriented Design

The platform follows a strict service-oriented architecture (SOA) where each component is independently deployable and horizontally scalable. The key design principles are:

- **Subsystem Registry** — Drop a new class implementing the Subsystem interface into the registry, and the API + UI automatically discover it with zero code changes elsewhere.
- **Mutable Configuration** — PlatformConfig is editable at runtime via PATCH /api/config; thresholds and ROI assumptions can be tuned per-customer without redeploying.
- **Versioned Model Registry** — FailureModelRegistry holds named, versioned models. The /api/models endpoint exposes the catalog and allows ops to promote versions.
- **Deterministic A/B Routing** — ABRouter uses hash(VIN) to ensure the same vehicle always routes to the same model arm, required for statistically valid A/B measurement.
- **Feedback Loop** — FeedbackStore captures customer annotations (true positive / false positive / missed) and triggers the retraining pipeline.

3.2 Component Layers

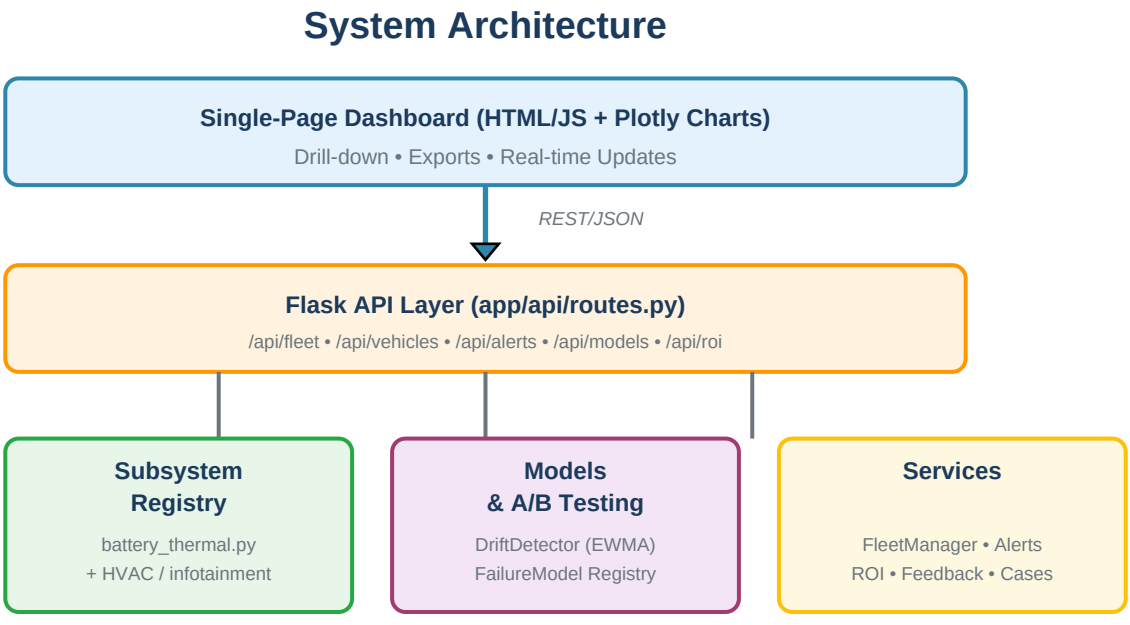

Figure 1: Service-Oriented Architecture — pluggable subsystems, versioned models, and independent services allow horizontal scaling without coupling.

Figure 1: Three-tier service-oriented architecture with pluggable subsystems.

4. Data & Telemetry Pipeline

4.1 Telemetry Simulation

The BatteryThermalSubsystem generates realistic synthetic telemetry that models real-world physics:

- **Drive cycle** — Drives current draw (0–250 A) which generates I^2R ohmic heating in the battery cells.
- **Cooling model** — Heat rejection is a function of coolant flow rate; this degrades when the pump is failing.
- **Digital twin (HPC expected)** — Assumes nominal hardware parameters to compute expected temperatures. The residual (actual − expected) is the core signal the platform exploits.

4.2 Digital Twin (HPC Expected vs. Actual)

The digital twin represents a high-fidelity physics simulation of the battery thermal system running on HPC infrastructure. For each timestep, it computes what the telemetry *should* read given the current operating conditions (ambient temperature, current draw, vehicle speed). The **residual** between the twin's prediction and actual field measurements is the fundamental anomaly signal:

```
residual = actual_measurement − twin_prediction
```

A healthy vehicle maintains near-zero residuals. A degrading pump causes positive residuals on temperature channels (less cooling → hotter than expected) and negative residuals on flow channels (reduced flow).

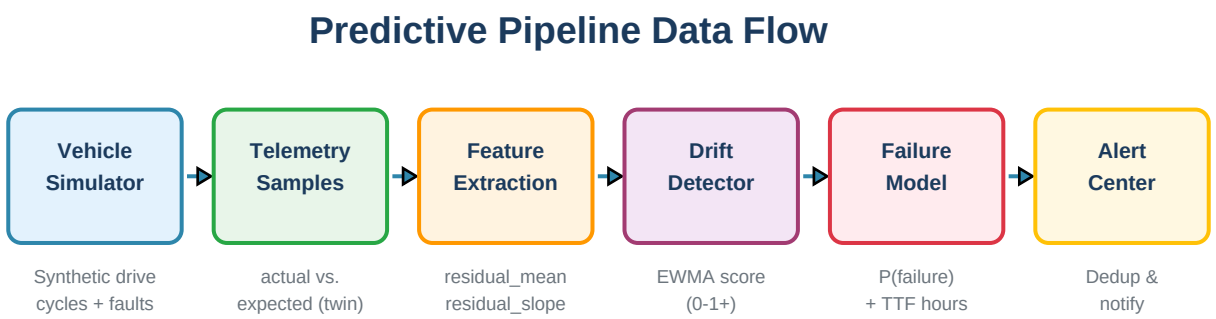

Figure 2: End-to-end flow from vehicle simulation through drift detection to actionable alerts.

Figure 2: End-to-end predictive pipeline from simulation to actionable alerts.

4.3 Failure Scenarios

Each simulated vehicle is assigned one of four failure scenarios using a deterministic seed (reproducible results). The scenarios model realistic degradation modes:

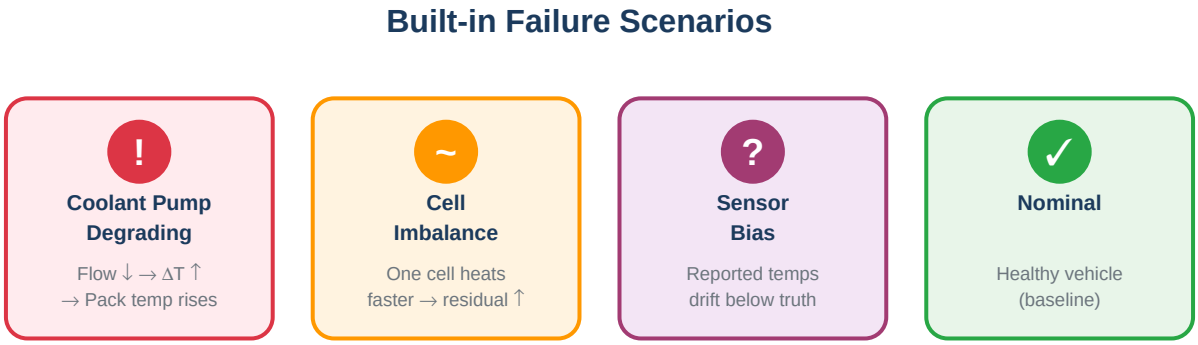

Figure 5: Each vehicle is randomly assigned a failure scenario (deterministic seed).

Figure 5: Four failure modes simulated per vehicle with deterministic seeding.

Scenario	Physical Effect	Detection Signal
`coolant_pump_degrading`	Flow slowly drops → ΔT widens → pack temperature	Temperature residual climbs above twin

`cell_imbalance`	One cell heats faster than the rest	Max-cell residual diverges from pack avg
`sensor_bias`	Reported temps drift below truth (hidden)	Caught via coolant ΔT residual mismatch
`nominal`	Healthy vehicle, normal operation	Residuals stay near zero

Table 2: Built-in failure scenarios with physical effects and detection signals.

5. Predictive Models

5.1 Feature Extraction

Raw telemetry samples are converted into an interpretable feature vector that feeds both the drift detector and the failure prediction model:

Feature	Formula	Interpretation
`residual_mean_*`	mean(actual – expected) over window	Average deviation per channel
`residual_slope_*`	linear slope of residuals over window	Rate of degradation (rising = worsening)
`drift_score`	EWMA-normalized composite	Overall anomaly magnitude (0–1+)

Table 3: Feature vector components derived from raw telemetry.

5.2 EWMA Drift Detection

The DriftDetector implements an **Exponentially Weighted Moving Average (EWMA)** approach that tracks per-feature statistics and produces a normalized drift score:

- **EWMA mean**: `mean_t = (1 − α) × mean_{t-1} + α × value_t` — tracks where the signal is deviating
- **EWMA variance**: `var_t = (1 − α) × (var_{t-1} + α × (value − mean)²)` — tracks instability
- **Score computation**: Residual-mean features contribute `|mean| / 3.0`; residual-slope features contribute `|mean| × 4.0`; averaged across channels
- **Alpha parameter**: Default $\alpha = 0.15$ (configurable), balancing responsiveness vs. noise smoothing

Alert thresholds:

- Drift score > 0.45 → **WARN** alert generated
- Drift score > 0.75 → **CRITICAL** alert generated

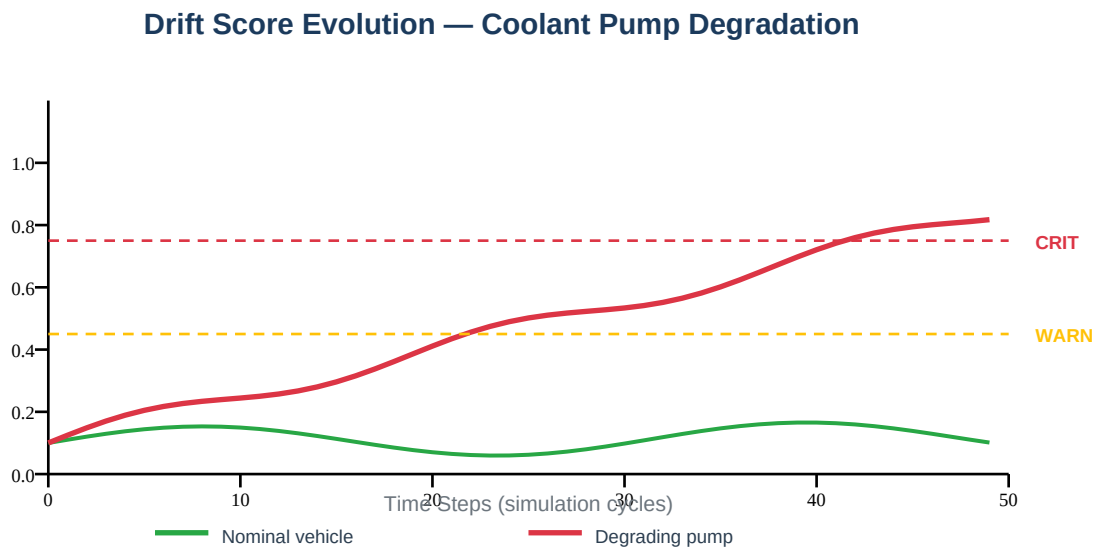

Figure 3: Drift score over time — healthy vs. degrading vehicle comparison.

5.3 Failure Prediction (Logistic Model)

The FailureModel uses a **transparent logistic regression** with named coefficients, allowing the dashboard to explain *why* a vehicle is flagged. The model outputs:

- **Failure probability** — sigmoid(z) where z = bias + Σ(weight_i × feature_i)
- **95% confidence interval** — Propagates epistemic uncertainty (σ) through the logit
- **Time-to-failure (TTF)** — Heuristic: base_hours = 72 × (1–p)², adjusted by slope signal. Range: 0.5h (imminent) to 72h (early warning)

Two shipped model versions:

Model	Version	Features Used	Characteristics
Baseline	baseline_v1	Residual means + drift score	Legacy thresholding approach; higher false negatives
EWMA+	ewma_plus_v2	Means + slopes + drift	Fires earlier; default active; better lead time

Table 4: Comparison of shipped failure prediction models.

5.4 A/B Testing Framework

The platform includes a production-grade A/B testing framework for comparing model versions in a statistically valid manner:

- **Hash-deterministic routing** — hash(VIN) mod 100 determines the arm. Same vehicle always sees the same model version, eliminating cross-contamination.
- **Configurable traffic split** — Default 70/30 (new/baseline), adjustable via API.
- **Per-arm metrics** — Precision, recall, and lead-time reported independently per arm.

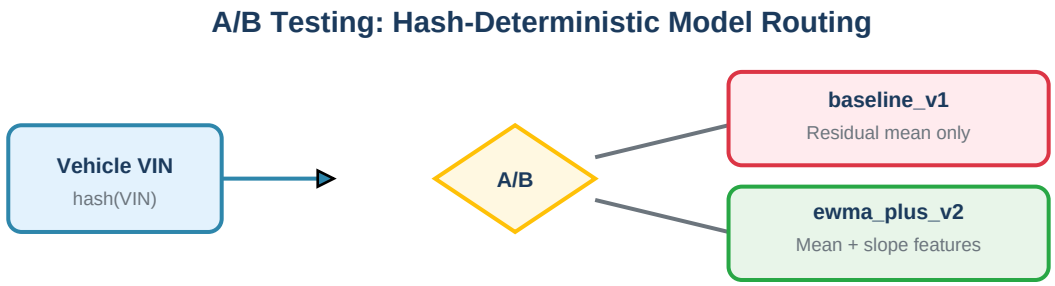

Figure 6: Same VIN always routes to same model arm for honest A/B measurement.

Figure 6: Hash-deterministic A/B routing ensures consistent model assignment per vehicle.

6. Dashboard & User Interface

The platform ships with two user interface options:

Standalone HTML Dashboard

A single self-contained HTML file (~88 KB) with **zero external dependencies**. All simulation, drift detection, models, alerts, and analytics run entirely in the browser via vanilla JavaScript and inline SVG charts. Features include:

- ■ / ■ / ■ simulation controls with 1×–30× speed
- Light / dark theme toggle
- Maintenance tab with TTF bucket grouping + service-window suggestions
- Fleet table sorting, free-text search, and multi-vehicle comparison overlay
- Fault injection controls for on-demand scenario testing
- Interactive ROI sliders with live recompute
- KPI sparkline history and toast notifications
- Snapshot import/export (JSON), fleet CSV export, print-to-PDF layout
- Shareable URLs via hash-encoded state + localStorage persistence

Flask-based Dashboard (Live Backend)

A multi-tab dashboard served by Flask with Plotly charts, connecting to the live simulation backend via REST API calls:

Tab	Purpose
Executive	C-level KPIs, projected warranty savings, validation-cycle reduction, detection accuracy
Fleet	Live fleet table with status badges, drift scores, failure probabilities, TTF, model assignment
Vehicle Drill-down	Per-vehicle digital twin (expected vs. actual), drift trend, load profile, alerts
Case Study	Reproducible scenario simulator comparing baseline vs. EWMA+ models side-by-side
Models & A/B	Versioned model registry, model activation, traffic split, per-arm precision/recall
Feedback	Customer intake form (TP/FP/missed) and on-demand retraining trigger
Config	Live edit of thresholds, ROI assumptions, and fleet size

Table 5: Dashboard tabs and their corresponding functionality.

7. REST API Reference

The platform exposes a comprehensive REST API for all platform operations. All endpoints return JSON unless otherwise specified.

Method	Path	Purpose
GET	`/api/health`	Liveness check
GET	`/api/subsystems`	List registered subsystems
GET/PATCH	`/api/config`	Read / patch platform configuration
GET	`/api/fleet`	Fleet summary + per-vehicle status
GET	`/api/vehicles/<vin>?window=N`	Telemetry timeline, twin, prediction
POST	`/api/fleet/reseed`	Re-initialise the simulated fleet
GET	`/api/alerts`	Alert feed
POST	`/api/alerts/<id>/ack`	Acknowledge an alert
GET	`/api/roi?fleet_size=...&detection_lift_pp=...`	ROI projection
GET	`/api/models`	Model registry catalog
POST	`/api/models/activate`	Activate a model version
PATCH	`/api/models/ab_split`	Update A/B traffic split
GET/POST	`/api/feedback`	Feedback submission & retrieval
POST	`/api/feedback/retrain`	Trigger model retraining
GET	`/api/case_study?scenario=...`	Reproducible scenario simulation
GET	`/api/reports/executive?format=json|csv`	Exportable management report

Table 6: Complete REST API endpoint reference.

8. ROI & Business Value

The platform includes a built-in ROI calculator that quantifies the business value of predictive maintenance for executive stakeholders. The `compute_roi(fleet_size, detection_lift_pct)` function produces:

- **Annual baseline vs. projected warranty cost**
- **Claims avoided per year** (based on detection improvement)
- **Validation-cycle savings** — default 35% reduction (within 30–40% target)
- **Total projected savings** used by the Executive dashboard tile

Configurable assumptions (editable via /api/config):

Parameter	Default Value	Description
Average claim cost	$12,000	Per-vehicle warranty claim cost for pack thermal failure
Claims per 1,000 vehicles/year	8.5	Baseline failure rate from historical data
Validation cycle cost	$2.4M/year	Engineering time spent on warranty validation
Recovery rate	35%	Percentage of claims prevented by early detection
Fleet size	50,000	Number of vehicles in monitored fleet

Table 7: ROI calculation parameters with default values.

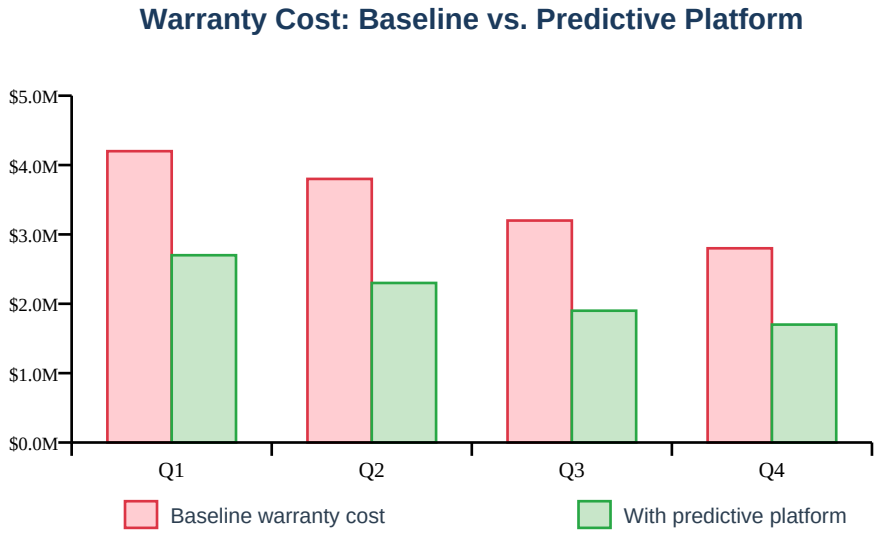

Figure 4: Projected quarterly warranty savings with 35% validation-cycle reduction.

Figure 4: Quarterly warranty cost comparison — baseline vs. predictive platform.

9. Extensibility

The platform is designed from day one for **horizontal extensibility**. Adding a new vehicle subsystem requires no changes to the dashboard, API layer, or predictive models.

Adding a New Subsystem (e.g., HVAC)

The process to add a new monitored subsystem follows three steps:

Step 1: Create a new file (e.g., `app/subsystems/hvac.py`) with a class implementing the `Subsystem` abstract interface: `reset()`, `step()`, `extract_features()`, plus metadata properties (`id`, `name`, `signals`, `units`).

Step 2: Register the subsystem in `app/subsystems/__init__.py`: `registry.register(HVACSubsystem())`

Step 3: (Optional) Point a second FleetManager instance at the new subsystem, or extend the existing one to multiplex.

The catalog endpoint (`/api/subsystems`) will automatically surface the new subsystem, and the dashboard will display it without any UI changes.

Key Design Strengths

- **Transparent models** — Named coefficients (not black-box neural nets). Dashboard explains why a vehicle is flagged.
- **Runtime configurability** — Thresholds, ROI assumptions, model activation — all mutable via API without redeployment.
- **Complete feedback loop** — Customer annotations → retraining trigger → new version → version history. Architecturally complete.
- **Zero-dependency demo** — Standalone HTML file runs the full simulation in-browser for instant sharing.
- **Production patterns** — A/B testing, model versioning, alert deduplication, confidence intervals — ready for production scale.

10. Getting Started

Instant Demo (Zero Setup)

Open `dashboard.html` directly in any modern browser. No server, no CDN, no installation required. The file is self-contained (~88 KB) and can be shared via email, USB, or any static hosting.

Flask Backend (Live HPC Integration)

For the full backend experience with live simulation:

```
pip install -r requirements.txt
python run.py
# Open http://localhost:5000
```

No external services required — the backend is pure Python + Flask with no numpy/sklearn dependency. Plotly is loaded from CDN in the browser.

Running Tests

```
python -m unittest discover tests -v
```

Repository Layout

Path	Purpose
`app/api/routes.py`	REST API endpoints
`app/config.py`	Mutable platform configuration
`app/main.py`	Flask application factory
`app/models/drift_detector.py`	EWMA-based drift scoring
`app/models/failure_predictor.py`	Logistic failure model + versioned registry
`app/services/ab_testing.py`	Hash-deterministic A/B router
`app/services/alerts.py`	Alert center with dedup & acknowledgement
`app/services/case_study.py`	Reproducible scenario harness
`app/services/feedback.py`	Customer feedback store + retrain log
`app/services/fleet_manager.py`	Background fleet simulator
`app/services/roi_calculator.py`	Warranty / validation savings
`app/static/`	Dashboard (index.html, app.js, style.css)
`app/subsystems/base.py`	Plugin interface (abstract class)
`app/subsystems/battery_thermal.py`	Shipped subsystem implementation

`app/subsystems/registry.py`	Pluggable catalog
`dashboard.html`	Standalone zero-dependency HTML demo
`run.py`	Convenience launcher (port 5000)
`tests/test_basic.py`	Smoke + integration tests

Table 8: Complete repository file structure with descriptions.